Schema-Aware Text-to-SQL

A Multi-Stage Retrieval & Dual Transformer Framework for Natural Language Database Queries

NLP Course Project Report

Mayank Tewatia (142301022) · Harsh (142301010) · Sai Varun (142301040)

1. Motivation

Relational databases are the backbone of modern information systems, powering everything from healthcare records and financial transactions to e-commerce platforms and enterprise analytics. Despite their ubiquity, accessing the information stored in these databases requires knowledge of SQL — a structured query language that remains inaccessible to the vast majority of end users.

Our project addresses this critical gap: the inability of non-technical users to interact with databases directly. Existing Text-to-SQL approaches suffer from one or more of the following limitations:

- No schema awareness: Early systems (Seq2SQL, SQLNet) were trained to map questions directly to SQL without incorporating database structure. They generalized poorly to unseen schemas.
- Full schema as input: State-of-the-art models (PICARD, DAIL-SQL, CHESS, MAC-SQL) feed the entire database schema to the model. For databases with hundreds of tables and thousands of columns, this leads to extremely long inputs, exceeding transformer context windows and degrading accuracy.
- No intermediate reasoning: All prior models jump directly from natural language to SQL without any intermediate, human-readable verification step. This makes them black boxes — if the SQL is wrong, there is no insight into why.

We chose to improve three specific aspects of the problem:

- Schema Retrieval: Replace full-schema input with intelligent, multi-signal retrieval of only the relevant tables and columns.
- Plan Generation: Introduce an intermediate execution plan (SCAN → FILTER → AGGREGATE → PROJECT) as an explicit reasoning step between the natural language question and the SQL output.
- Generalizability: Ensure the system works on any new database without retraining or hardcoding any table or column names.

Our target applications include conversational database interfaces, healthcare record querying systems, and enterprise analytics tools — any scenario where non-technical users need to query structured data interactively.

2. Approach

Our system consists of two major components: (1) a Multi-Stage Schema Retriever that selects relevant schema elements, and (2) Transformer-based generation models that produce an execution plan and SQL query. We implemented and compared two architectures for the generation step.

2.1 Multi-Stage Schema Retriever

The retriever takes the full database schema (as a .sql file) and a natural language question as input, and outputs a compact, question-relevant schema string with foreign key connections. It operates through five progressive stages:

Stage 1: Two-Signal Fusion Scoring

Each table in the database is scored using two complementary signals. Note: an earlier design of the retriever included four signals (Bi-encoder, BM25, Column Mention, and Value Pattern). In the current implementation, Column Mention scoring and Value Pattern scoring were removed to simplify the pipeline and reduce computational overhead.

- Bi-encoder (dense retrieval): Encodes the question and each table description into dense vector representations using a pre-trained sentence transformer. Computes cosine similarity to capture semantic relevance. Formally: score_bi(q, t) = cos(E_q(q), E_t(t)), where E_q and E_t are the question and table encoders. Base weight W_BIENCODER_BASE = 0.40.
- BM25 (lexical retrieval): Computes term frequency-inverse document frequency overlap between question tokens and table and column name tokens. Uses the entire table corpus to compute proper IDF weights, making it more robust than naive token overlap. Base weight W_BM25_BASE = 0.25.

The final fusion score is:

```
score_fusion(q, t) = W_BIENCODER_BASE x score_bi(q, t) + W_BM25_BASE x score_BM25(q, t)
```

The top-k tables (up to MAX_TABLES = 5) are selected by fusion score. Junction tables not mentioned in the question receive a penalty of JUNCTION_PENALTY = -0.15.

LIMITATION — No hyperparameter tuning was performed for the retriever. The values of W_BIENCODER_BASE (0.40), W_BM25_BASE (0.25), DROP_RATIO (0.65), GAP_RATIO (0.25), MAX_TABLES (5), CE_WEIGHT (0.60), FUSION_WEIGHT (0.40), and JUNCTION_PENALTY (-0.15) were chosen by querying a large language model (Claude) for reasonable starting defaults. No empirical validation, grid search, or ablation was performed. Systematic hyperparameter tuning is planned as a key area of future work.

Stage 2: Cross-Encoder Reranking

The top-6 candidates are re-scored using a more powerful joint model. We use the MS-MARCO cross-encoder, which processes (question, table_description) pairs jointly rather than independently:

```
score_CE(q, t) = CrossEncoder([CLS] q [SEP] desc(t) [SEP])
```

The final combined score is:

```
score_final(q, t) = 0.6 × score_CE(q, t) + 0.4 × score_fusion(q, t)
```

This two-stage approach achieves the best of both worlds: fast approximate recall from Stage 1 and precise reranking from the cross-encoder.

Stage 3: Adaptive Threshold Selection

Instead of a fixed top-k, we use a ratio-based cutoff with a gap guard. The list is cut at rank k where the score drops significantly:

```
cut at k where score(k+1) / score(1) < DROP_RATIO (= 0.65)
```

A gap guard prevents cutting when all scores are close together, which would incorrectly discard relevant tables in ambiguous queries. This yields 1 to 5 selected tables per query.

Stage 3.5: FK Neighbor Expansion

From each selected table, we walk one hop along the foreign key (FK) graph. A neighboring table is added only if its name or column names semantically match the question. This prevents over-inclusion of irrelevant junction tables.

Stage 4: Bridge BFS

When two selected tables are not directly connected by a foreign key, we run Breadth-First Search (BFS) on the FK graph to find the shortest connecting path. All intermediate (bridge) tables along this path are added to enable JOIN-able schema. This process repeats iteratively until the selected table set is stable.

Stage 4.5: Column Pruning

For each selected table, we prune its columns to keep only those that are:

- Structural: Primary keys (PK) and Foreign keys (FK)
- Text-matched: Column names that appear in the question tokens
- Value-matched: Columns whose values match question terms
- Superlative-matched: Columns relevant to superlative expressions (max, min, oldest, youngest)
- Cross-encoder scored: Columns that the cross-encoder assigns high relevance to

Stage 5: Format Output

The final output is formatted as a structured string:

```
question: ... | schema: table1 ( col1 [PK], col2 ) | foreign keys: table1.col2 ->
table2.col3
```

2.2 Generation Models

We implemented and compared two transformer-based architectures for generating the execution plan and SQL query from the retrieved schema.

Method 1: Single Transformer (End-to-End)

The single transformer receives the concatenated question and pruned schema as input, and produces both the plan and SQL in a single decoding pass:

Input: `question: "Find the names of all singers who performed in a concert in 2014" | schema: singer(singer_id, name), concert(concert_id, year), singer_in_concert(singer_id, concert_id) | foreign keys: singer_in_concert.singer_id = singer.singer_id, singer_in_concert.concert_id = concert.concert_id`

Output:

```
Step 01  [SCAN]       Sequential scan on table 'singer_in_concert' AS 'T1'
Step 02  [SCAN]       Sequential scan on table 'singer' AS 'T2'
Step 03  [HASH-JOIN]  INNER JOIN T2 ON T1.singer_id = T2.singer_id
Step 04  [SCAN]       Sequential scan on table 'concert' AS 'T3'
Step 05  [HASH-JOIN]  INNER JOIN T3 ON T1.concert_id = T3.concert_id
Step 06  [FILTER]     Apply WHERE: T3.year = 2014
Step 07  [PROJECT]    SELECT T2.name
||
SELECT T2.name FROM singer_in_concert T1
JOIN singer T2 ON T1.singer_id = T2.singer_id
JOIN concert T3 ON T1.concert_id = T3.concert_id
WHERE T3.year = 2014;
```

We use a T5 (Text-to-Text Transfer Transformer) architecture. The training objective is cross-entropy loss over the combined plan+SQL token sequence:

```
L = -Σ log P(y_i | y_<i, x)  for all tokens y_i in [plan || sql]
```

The plan and SQL are separated by a special delimiter token (||) during training and inference.

Method 2: Dual Transformer (Sequential Reasoning)

The dual transformer uses two separate T5 models trained independently:

- Transformer 1 (Planner): Trained on (Question + Schema) → Plan pairs only.
- Transformer 2 (SQL Generator): Trained on (Question + Schema + Plan) → SQL pairs.

At inference time, the output of Transformer 1 is fed as additional context to Transformer 2:

T1 Input: `question: "Find the names of all singers who performed in a concert in 2014" | schema: singer(singer_id, name), concert(concert_id, year), singer_in_concert(singer_id, concert_id)`

T1 Output (Plan):

```
Step 01  [SCAN]       Sequential scan on table 'singer_in_concert' AS 'T1'
Step 02  [SCAN]       Sequential scan on table 'singer' AS 'T2'
Step 03  [HASH-JOIN]  INNER JOIN T2 ON T1.singer_id = T2.singer_id
Step 04  [SCAN]       Sequential scan on table 'concert' AS 'T3'
Step 05  [HASH-JOIN]  INNER JOIN T3 ON T1.concert_id = T3.concert_id
Step 06  [FILTER]     Apply WHERE: T3.year = 2014
Step 07  [PROJECT]    SELECT T2.name
```

T2 Input (Plan only):

```
Step 01  [SCAN]       Sequential scan on table 'singer_in_concert' AS 'T1'
Step 02  [SCAN]       Sequential scan on table 'singer' AS 'T2'
Step 03  [HASH-JOIN]  INNER JOIN T2 ON T1.singer_id = T2.singer_id
Step 04  [SCAN]       Sequential scan on table 'concert' AS 'T3'
Step 05  [HASH-JOIN]  INNER JOIN T3 ON T1.concert_id = T3.concert_id
Step 06  [FILTER]     Apply WHERE: T3.year = 2014
Step 07  [PROJECT]    SELECT T2.name
```

T2 Output (SQL):

```sql
SELECT T2.name FROM singer_in_concert T1
JOIN singer T2 ON T1.singer_id = T2.singer_id
JOIN concert T3 ON T1.concert_id = T3.concert_id
WHERE T3.year = 2014;
```

Each model is trained with cross-entropy loss over its respective target sequence. This decomposition allows each model to focus on a single, well-defined sub-task, reducing gradient interference.

Intermediate Execution Plan Format

The execution plan follows a structured format inspired by relational algebra:

Question: "Find the average age of employees in the Engineering department"

step1: SCAN	table: employees → full scan, load all rows
step2: FILTER	condition: department = 'Engineering' → keep rows matching predicate
step3: AGGREGATE	type: GROUP_ALL → compute AVG(age) → reduce filtered rows to one value
step4: PROJECT	columns: AVG(age) → output only requested column

The plan vocabulary maps naturally to SQL clauses: SCAN → FROM, FILTER → WHERE, AGGREGATE → GROUP BY/aggregation functions, PROJECT → SELECT. JOIN steps are added for multi-table queries.

3. Data

3.1 Spider Benchmark

The primary dataset used is the Spider Text-to-SQL benchmark (Yu et al., 2018). Key statistics:

- 10,181 question-SQL pairs across 200 complex databases
- 138 distinct domains (academic, finance, healthcare, sports, etc.)
- Multi-table queries requiring JOINs, nested subqueries, aggregations, and GROUP BY clauses
- Train / Development split: 7,000 / 1,034 examples

Spider is the de-facto standard benchmark for cross-domain Text-to-SQL evaluation and allows direct comparison with prior published systems.

3.2 Synthetic Dataset (Newly Generated)

We additionally constructed and evaluated on a newly generated synthetic dataset designed specifically to test generalization to unseen schemas. Key properties:

- Diverse table structures and column types (numeric, categorical, date, boolean)
- Database schemas not seen during training — true zero-shot generalization test
- Covers query types including aggregation, filtering, multi-table joins, and ordering

This dataset evaluates whether the system can be deployed on a new database without any fine-tuning or schema-specific adjustments.

3.3 External Resources

- MS-MARCO Cross-Encoder: Used as the reranking model in Stage 2 of the retriever (pre-trained, not fine-tuned by us).
- T5 Pre-trained Weights: Used as the base model for both transformer architectures (huggingface transformers library).
- Spider Database Files: Official .sql schema files from the Spider dataset repository.

4. Code

The following external code and libraries were used in this project:

- Hugging Face Transformers v4.40.0 (https://github.com/huggingface/transformers): Used for Flan-T5-XL model loading, tokenization, and inference.
- PEFT v0.10.0 (https://github.com/huggingface/peft): Used for LoRA (Low-Rank Adaptation) fine-tuning. PeftModel.from_pretrained used to load and apply LoRA adapter at inference time.
- Accelerate v0.29.3 (https://github.com/huggingface/accelerate): Used for distributed training and GPU device management.
- Sentence-Transformers (https://github.com/UKPLab/sentence-transformers): Used for the bi-encoder (all-MiniLM-L6-v2) and MS-MARCO cross-encoder in the retriever pipeline.
- BM25Okapi from rank_bm25 (https://github.com/dorianbrown/rank_bm25): Used for lexical scoring in Stage 1 of the retriever.
- Spider Dataset Repository (https://yale-lily.github.io/spider): Official dataset files, schema definitions, and evaluation scripts.
- Hugging Face Evaluate Library: Used to compute token-level F1 scores for plan and SQL prediction evaluation.

All retriever logic (multi-signal fusion, adaptive thresholding, FK neighbor expansion, Bridge BFS, column pruning, and output formatting) was implemented from scratch by our group.

Both transformer training pipelines (Single Transformer and Dual Transformer) were implemented by our group using the Hugging Face Trainer API with default hyperparameter configurations.

5. LLMs Used

We used the following language models in our project:

5.1 Flan-T5-XL with LoRA (Primary Generation Model)

- Model: google/flan-t5-xl (encoder-decoder architecture, 3B parameters, instruction-tuned variant of T5)
- Fine-tuning method: Low-Rank Adaptation (LoRA) via the PEFT library (peft==0.10.0). LoRA inserts trainable low-rank matrices into the attention layers of the frozen base model, reducing trainable parameters by over 99% compared to full fine-tuning.
- Usage: Fine-tuned as the backbone for both the Single Transformer and Dual Transformer generation models.

- Training: Supervised fine-tuning on (question + retrieved schema) -> (plan + SQL) sequences. Cross-entropy loss over full output token sequences. LoRA adapter saved as lora_adapter_combined.
- Inference: Beam search with num_beams=4, early_stopping=True, length_penalty=1.0, max_new_tokens=512, max_input_length=512.
- Libraries: transformers==4.40.0, peft==0.10.0, accelerate==0.29.3, torch with float16/bfloat16 precision.

5.2 MS-MARCO Cross-Encoder

- Model: cross-encoder/ms-marco-MiniLM-L-6-v2 (from sentence-transformers)
- Usage: Used in Stage 2 of the retriever for reranking candidate tables.
- Not fine-tuned — used directly with pre-trained weights for relevance scoring of (question, table_description) pairs.
- No prompting required — used as a scoring function, not a generative model.

5.3 Bi-Encoder

- Model: sentence-transformers/all-MiniLM-L6-v2
- Usage: Semantic similarity scoring in Stage 1 of the retriever.
- Not fine-tuned — used off-the-shelf for dense vector representations.

Note on LLM usage: We used Claude (a large generative LLM) to suggest initial hyperparameter values for the retriever (see Section 6.3). No LLMs were used for SQL generation or evaluation inference — all generation is performed by our fine-tuned Flan-T5-XL models.

6. Experimental Setup

6.1 Evaluation Metrics

We evaluate using Token-level F1 scores for all generation components. Token F1 is chosen over exact match accuracy because:

- Exact match is binary (0 or 1) and fails to distinguish partial correctness from complete failure.
- Semantically equivalent SQL queries can differ in token sequence (e.g., column ordering, spacing, alias naming).
- Token F1 captures both over-generation (high recall, low precision) and under-generation (high precision, low recall) — providing a more informative evaluation signal.

Formally, for a predicted sequence P and gold sequence G:

```
Precision = |tokens(P) ∩ tokens(G)| / |tokens(P)|
Recall    = |tokens(P) ∩ tokens(G)| / |tokens(G)|
F1        = 2 × (Precision × Recall) / (Precision + Recall)
```

We report the following metrics:

Metric	Component	What It Measures
Plan Token F1	Generator	Token overlap between predicted plan and gold plan
SQL Token F1	Generator	Token overlap between predicted SQL and gold SQL
Table F1	Retriever	Correct tables retrieved vs. gold required tables
Column F1	Retriever	Correct columns retrieved vs. gold required columns
FK F1	Retriever	Correct FK connections retrieved vs. gold

6.2 Conditions Compared

We evaluated two experimental conditions for the generation models:

- Gold Schema Condition: The transformer models receive the ground-truth schema (exactly the tables and columns needed), bypassing the retriever. This establishes an upper-bound on generation quality.
- Retriever Schema Condition: The transformer models receive the schema as retrieved by our 5-stage pipeline. This evaluates the full end-to-end system.

For each condition, we compare:

- Single Transformer (Method 1): One T5 model producing plan + SQL jointly.
- Dual Transformer (Method 2): Two T5 models — one for plan generation, one for SQL generation using the generated plan as input.

6.3 Hyperparameters

No hyperparameter tuning was performed. All models were trained using default values from the Hugging Face Seq2SeqTrainer:

- Optimizer: AdaFactor (default for T5)
- Learning rate: Default schedule
- Batch size: Default (8 per device)
- Max sequence length: 512 tokens
- Training epochs: 10

Using default hyperparameters demonstrates that our architectural design choices (retrieval + plan generation) are robust and not dependent on cherry-picked training configurations.

6.4 Datasets

- Spider Benchmark: Standard cross-domain Text-to-SQL benchmark (train/dev split)
- Synthetic Dataset: Newly generated dataset with unseen schemas for zero-shot generalization testing

7. Results

7.1 Retriever Performance

Table 1 shows retriever performance on the test set. These results measure how accurately the retriever identifies the necessary schema elements before passing them to the generation model.

Component	Metric	Score (%)
Table Selection	Table F1	75.58
Column Selection	Column F1	68.52
FK Connections	FK F1	70.66

Table 1: Retriever component-level F1 scores.

7.2 Generator Results — Gold Schema Condition

Table 2 presents results when both models receive the gold (ground-truth) schema. This isolates the generation quality from retriever imperfections.

Model	Dataset	Plan Token F1 (%)	SQL Token F1 (%)
Single Transformer	Synthetic (New)	90.01	90.99
Single Transformer	Spider	93.93	94.02
Dual Transformer	Synthetic (New)	90.74	92.32
Dual Transformer	Spider	**95.18 ★**	**94.80 ★**

Table 2: Generation results with gold schema input. ★ = best score.

7.3 Generator Results — Retriever Schema Condition (Full System)

Table 3 presents results for the complete end-to-end pipeline, where the retriever's output is fed to the generation model. These results reflect real-world deployment performance.

Model	Dataset	Plan Token F1 (%)	SQL Token F1 (%)
Single Transformer	Spider	84.95	84.68
Dual Transformer (T1: Plan)	Spider	83.99	—
Dual Transformer (T2: SQL)	Spider	—	83.96

Table 3: End-to-end system results with retriever schema input.

7.4 Summary Comparison: Gold Schema vs. Retriever Schema

Table 4 directly compares generation quality under gold versus retriever schema conditions to quantify the retrieval gap.

Model	Condition	Plan F1 (%)	SQL F1 (%)	SQL Drop
Single Transformer	Gold Schema (Spider)	93.93	94.02	—
Single Transformer	Retriever Schema	84.95	84.68	-9.34 pts
Dual Transformer	Gold Schema (Spider)	95.18	94.80	—
Dual Transformer	Retriever Schema	83.99	83.96	-10.84 pts

Table 4: Impact of retriever on generation quality.

8. Analysis of Results

8.1 Did We Improve Over the Baseline?

Our work does not directly compare against a single fixed baseline, but we position ourselves relative to the prior literature and establish our own internal baselines. Here is our analysis:

Gold Schema Results: Strong Generation Quality

Under the gold schema condition, both models achieve strong F1 scores:

- Single Transformer achieves 94.02 SQL Token F1 on Spider with no hyperparameter tuning.
- Dual Transformer achieves 94.80 SQL Token F1 on Spider — a 0.78-point improvement over the Single Transformer.
- The Dual Transformer also achieves the best Plan Token F1 of 95.18 on Spider.

These results confirm that plan-conditioned SQL generation (Dual Transformer) outperforms joint generation (Single Transformer) by small margin . Task decomposition works: each model has a focused objective, leading to cleaner gradient signals and better output quality.

Retriever Results: Moderate Performance with Room to Grow

The retriever achieves Table F1 of 75.58, Column F1 of 68.52, and FK F1 of 70.66. This is a solid first result for a system built without any hardcoded schema knowledge. Column-level retrieval is the weakest component because:

- Each table may have 5-20+ columns, making column selection a high-dimensional problem.
- Column names are often abbreviated or domain-specific, making semantic matching harder.
- Value-pattern matching helps but is limited to numeric and simple string patterns.

Full System (Retriever + Generator): Gap Analysis

The full end-to-end system shows a performance drop compared to the gold schema condition:

- Single Transformer: SQL F1 drops from 94.02 (gold) to 84.68 (retriever) — a gap of 9.34 points.
- Dual Transformer: SQL F1 drops from 94.80 (gold) to 83.96 (retriever) — a gap of 10.84 points.

This gap is expected and attributable to retriever errors:

- When the retriever misses a required table (Table F1 = 75.58), the generator cannot produce the correct JOIN.
- When required columns are missed (Column F1 = 68.52), the generator may hallucinate column names or produce incorrect SELECT clauses.
- FK retrieval errors (FK F1 = 70.66) cause missing JOIN conditions in multi-table queries.

Importantly, despite this gap, our system achieves SQL F1 scores in the 84-85% range without any hyperparameter tuning, using only default configurations. This demonstrates that the core architectural design is sound.

Why the Dual Transformer Gap is Slightly Larger

Under the retriever condition, the Dual Transformer (83.96) performs slightly below the Single Transformer (84.68) in SQL F1. This is due to error propagation:

- If the plan generated by T1 contains an error caused by a bad schema (missing table or column), T2 receives an incorrect plan as input.
- T2 then generates SQL conditioned on this wrong plan, compounding the error.
- The Single Transformer, generating plan and SQL jointly, has a single error source rather than a cascading chain.

Under gold schema conditions, this effect disappears and the Dual Transformer performs better — confirming that error propagation is a retrieval-quality issue, not a fundamental architectural weakness.

8.2 Comparison to Prior Work

While direct numerical comparison is difficult without identical evaluation settings, our approach offers qualitative advantages over prior work:

- DIN-SQL and DAIL-SQL: Use full schema as input and achieve high exact match scores, but cannot scale to large databases without exceeding context limits. Our retrieval-first approach addresses this.
- CHESS: Uses a retrieval-based approach but does not generate intermediate execution plans. Our plan generation adds interpretability absent from CHESS.
- MAC-SQL: Uses multi-agent LLMs for decomposition. Our approach achieves comparable reasoning quality using smaller, fine-tuned transformers — significantly reducing inference cost.
- No prior approach in our reference set combines both 5-stage retrieval and intermediate plan generation. This is our unique contribution.

9. Future Work

9.1 Immediate Improvements

- Hyperparameter Tuning: All results in this project use default hyperparameter values. Systematic tuning of learning rate, batch size, number of epochs, and sequence length could push F1 scores significantly higher. We expect improvements of 2-5 points from tuning alone.
- Stronger Column Retrieval: Column F1 (68.52) is the weakest component and the largest contributor to the retriever-generator gap. Future work should explore value-level semantic matching, column description augmentation, and fine-tuning the cross-encoder specifically for column relevance scoring.
- Execution Feedback Loop: Implement SQL execution against the database after generation. If execution fails (syntax error or empty result), use the error as feedback to refine the plan and regenerate SQL. This would close the verification loop and allow self-correction.

9.2 Evaluation Extensions

- Exact Match Evaluation: Compare with Exact Match (EM) accuracy to align with the Spider leaderboard and enable direct comparison with published results.
- Execution Accuracy: Measure whether the generated SQL, when executed against the database, produces the correct result set. This is the most meaningful real-world metric.
- Error Analysis: Categorize failure modes (wrong table, wrong join, wrong aggregation, wrong filter) to identify the most impactful areas for improvement.
- Non-English Queries: Test generalization to non-English natural language questions, which is critical for global deployment.

10. References

- Yu, T., Zhang, R., Yang, K., Yasunaga, M., Wang, D., Li, Z., ... & Radev, D. (2018). Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. EMNLP 2018.
- Gao, D., Wang, H., Li, Y., Sun, X., Qian, Y., Ding, B., & Zhou, J. (2023). Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation. arXiv:2308.15363. (DAIL-SQL)
- Pourreza, M., & Rafiei, D. (2023). DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction. NeurIPS 2023.
- Guo, J., Zhan, Z., Gao, Y., Xiao, Y., Lou, J.G., Liu, T., & Zhang, D. (2019). Towards Complex Text-to-SQL in Cross-Domain Database with Intermediate Representation. ACL 2019. (IRNet)
- Wang, B., Shin, R., Liu, X., Polozov, O., & Richardson, M. (2020). RAT-SQL: Relation-Aware Schema Encoding and Linking for Text-to-SQL Parsers. ACL 2020.
- Scholak, T., Schucher, N., & Bahdanau, D. (2021). PICARD: Parsing Incrementally for Constrained Auto-Regressive Decoding from Language Models. EMNLP 2021.

- Wang, B., Ren, G., Chen, J., & Zhao, X. (2023). MAC-SQL: A Multi-Agent Collaborative Framework for Text-to-SQL. arXiv:2312.11242.
- Gao, Q., Wang, B., Ye, J., Chen, J., & Zhao, X. (2024). CHESS: Contextual Harnessing for Efficient SQL Synthesis. arXiv:2405.16755.
- Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., ... & Liu, P.J. (2020). Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. JMLR 2020. (T5)
- Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP 2019. (Sentence-Transformers)